The National Higher School of Artificial Intelligence
Natural Language Processing
Final Exam 22/1/2025
Duration: 2 hours

Exercise 1		**/ 5.5**
Exercise 2		**/ 6**
Exercise 3		**/ 4.5**
Exercise 4		**/ 4**
TOTAL		**/20**

Exercise 1: **5.5 points** **Estimated time 25 minutes**
MCQs **5.5 points (all questions out of 0.25 each, except question 5 which is out of 0.5) So potential total is 5.75**

Answer the following multiple choice questions by circling the correct answer. All questions are out of 0.5 each, except question 5 which is out of 0.5. A correct answer gets the allocated mark for the question; half the mark will be deducted for each incorrect answer.

Part I. Suppose you have the following training data for Naïve Bayes:

```
I liked the movie [LABEL=+]
I hated the movie because it was an action movie [LABEL=-]
Really cool movie [LABEL=+]
```

(1) What is the unsmoothed maximum likelihood estimate of P(+) for this data?
A. 1/3
B. 1/2
C. 2/3
D. 1

Answer: C, 2/3 (two positive / one negative) 0.25 points

(2) What is the unsmoothed maximum likelihood estimate of P("movie"|+) for this data?
A. 2/17
B. 1/5
C. 2/7
D. 1/2
Answer: C, 2/7 (7 words emitted in positive samples, 2 of them are movie) 0.25 points

(3) Suppose we are given an unseen input sentence the movie. What is the joint probability P(−, "the movie")?
A. 2/300
B. 4/98
C. 1/12
D. 1/3
Answer: A, 0.25 points
P(-|"the movie") = P(-) * P("the"| -) * P("movie"| -) = 1/3 * 1/10 * 2/10 = 2/300 = 1/150

(4) What prediction will the model make on "the movie"?
A. Positive
B. Negative
C. Cannot decide

Answer: A, 0.25 points
P(+|"the movie") = P(+) * P("the"| +) * P("movie"| +) = 2/3 * 1/7 * 2/7 = 4/147
P(-|"the movie") = P(-) * P("the"| -) * P("movie"| -) = 1/3 * 1/10 * 2/10 = 2/300 = 1/150

So the sentiment is *Positive*

Part II: All the remaining questions.

(5) Assume the following simplified word vectors:

- girl → vector g = (1,3)
- boy → vector b = (2, 1)
- princess → vector p = (4, 5)
- king → vector k = (6,2)
- queen → vector q = (3,6)

Using cosine similarity, which word is closest to girl?

A. boy
B. princess
C. king
D. queen

Answer: D, 0.5 points

Then the length of
g is ||g|| = sqrt(1**2 + 3**2) = sqrt(10)
b is ||b||= sqrt(5)
p is ||p|| = sqrt(4**2 + 5**2) = sqrt(41)
k is ||k|| = sqrt(6**2 + 2**2) = sqrt(40)
q is ||q|| = sqrt(3**2 + 6**2) = sqrt(45)
So
cos(g, b) = (g dot b)/(||g||*||b||) = (1*2 + 3*1) / (sqrt(10)*sqrt(5)) = 5 /sqrt(50) = 1/sqrt(2) = 0.71
cos(g, p) = (g dot p)/(||g||*||p||) = (1*4 + 3*5)/(sqrt(10)*sqrt(41)) = 19 /sqrt(410) = 0.93
cos(g, k) = (g dot k)/(||g||*||k||) = (1*6 + 3*2)/(sqrt(10)*sqrt(40)) = 12 /sqrt(400) = 0.85
cos(g, q) = (g dot q)/(||g||*||q||) = (1*3+ 3*6)/(sqrt(10)*sqrt(45)) = 21 /sqrt(450) = 0.99

Thus *queen* is closest to girl.

(6) Which of the following regular expressions does not match a string containing a valid email address?

A. ^[a-zA-Z0-9]([a-zA-Z0-9]+[\._%+-])+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$
B. \w+@\w+\.\w+
C. ^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,6}$
D. [a-zA-Z0-9]+@[a-zA-Z]+\.[com|org|net]
E. None of the above.

Answer: E) 0.25 points
Explanation: It is easy to check that none of them does

(7) Consider the problem of tokenizing a sentence that contains contractions such as "don't". If you want to preserve the contraction as a single token, which regular expression would be most appropriate for tokenizing the sentence?

A. \w+|\'\w+
B. \w+|\S+
C. [a-zA-Z]+(-[a-zA-Z]+)?

D. \w+|\S+|\'\w+

Answer: A) \w+|\'\w+ 0.25 points

(8) In the context of minimum edit distance, if you wanted to calculate the edit distance between two strings, but you want to minimize the cost of transposition operations (swap of adjacent characters), which of the following modifications would you make to the standard edit distance algorithm?

A. Increase the cost of insertions and deletions
B. Make the cost of a substitution operation the same as that of insertion and deletion
C. Add an additional operation for transpositions with lower cost
D. Increase the cost of matching characters

Answer: C) Add an additional operation for transpositions with lower cost
Explanation: The standard edit distance (Levenshtein distance) does not account for transpositions. To minimize the cost of transposition, you would add an operation with lower cost to the algorithm.
0.25 points

(9) Which of the following would increase the likelihood of overfitting in an n-gram language model?

A. Using more data for training the model
B. Increasing the size of the n-grams (e.g., using a 5-gram instead of a 3-gram model)
C. Applying smoothing techniques
D. Using lower-order n-grams (e.g., bigrams) rather than higher-order n-grams

Answer: B) Increasing the size of the n-grams (e.g., using a 5-gram instead of a 3-gram model)
Explanation: Larger n-grams capture more specific contexts, and as a result, they are more likely to overfit the training data by memorizing rare combinations. This can lead to poor generalization to unseen data.

(10) In evaluating the performance of an n-gram language model using perplexity, which of the following decreases perplexity?

A. Using a larger n-gram (e.g., from bigrams to 3-grams)
B. Increasing the size of the training corpus without changing the model
C. Reducing the size of the training corpus to remove noisy data
D. Applying heavier smoothing methods that assign more probability to unseen n-grams

Answer: B) Increasing the size of the training corpus without changing the model
Explanation: Increasing the size of the training corpus helps the model better capture the distributions of word sequences, leading to a more accurate model and lower perplexity. It improves the model's ability to generalize to unseen data.

(11) In the context of word embeddings such as Word2Vec, which of the following best describes the process of negative sampling?

A. Replacing the rare words with a zero vector to reduce noise
B. Using random word pairs from the corpus to speed up training
C. Using the most frequent words in the corpus as negative examples to reduce the impact of rare words
D. Using a set of randomly selected words as negative examples to improve training efficiency

Answer: D) Using a set of randomly selected words as negative examples to improve training efficiency
Explanation: Negative sampling improves the training efficiency by randomly selecting negative examples (words that do not appear in the context of the target word) to update the model's weights.

(12) In neural language models, word embeddings serve as input representations for words. Which of the following most likely improves the quality of these embeddings during training?

A. Reducing the size of the hidden layers
B. Using larger training corpora and more diverse contexts
C. Training the model with small batch sizes
D. Using low-dimensional embeddings for words

Answer: B) Using larger training corpora and more diverse contexts
Explanation: Larger and more diverse corpora provide better context for learning high-quality word embeddings, as they capture a wider range of word co-occurrences and syntactic/semantic relationships.

(13) In a stacked RNN architecture, what is the main advantage of using multiple layers of RNNs?

A. The model becomes less prone to overfitting by using more parameters.
B. Stacking RNNs helps the model capture more complex hierarchical dependencies in the input sequences.
C. It simplifies training by reducing the number of computational resources required.
D. It prevents the model from suffering from the vanishing gradient problem.

Answer: B) Stacking RNNs helps the model capture more complex hierarchical dependencies in the input sequences.
Explanation: Stacked RNNs allow the model to learn representations at different levels of abstraction, improving its ability to capture complex patterns in sequential data.

(14) Bidirectional RNNs are commonly used for tasks where the context from both past and future is important. What is the main disadvantage of using a bidirectional RNN in NLP?

A. It cannot handle long-term dependencies effectively.
B. It requires more computational resources due to the need for two separate RNNs.
C. It requires larger training sets to avoid overfitting.
D. It is only effective for classification tasks.

Answer: B) It requires more computational resources due to the need for two separate RNNs.
Explanation: Bidirectional RNNs have two separate RNNs: one processing the sequence forward and one backward, which doubles the computational load compared to a unidirectional RNN.

(15) In the multi-head attention mechanism of the Transformer, what is the main purpose of having multiple attention heads?

A. To allow the model to attend to different parts of the input sequence with different attention weights.
B. To speed up the computation by processing multiple sequences in parallel.
C. To reduce the size of the model by sharing weights across heads.
D. To create a more uniform attention distribution across all tokens in the sequence.

Answer: A) To allow the model to attend to different parts of the input sequence with different attention weights.
Explanation: Multi-head attention enables the model to focus on different relationships and patterns within the input sequence by using multiple attention heads, each learning a different representation.

(16) One of the challenges of pretraining large language models like GPT-3 is the massive computational cost. Which of the following approaches can mitigate this cost while maintaining performance?

A. Using smaller training corpora for faster convergence during pretraining

B. Training on specialized hardware such as TPUs or GPUs optimized for parallel processing
C. Reducing the number of layers in the Transformer architecture
D. Reducing the model size by using fewer attention heads during training

Answer: B) Training on specialized hardware such as TPUs or GPUs optimized for parallel processing
Explanation: Specialized hardware like TPUs and GPUs allows for the parallelization of matrix operations, significantly speeding up the training process for large-scale models.

(17) In pretraining large language models like BERT, why is it beneficial to use masked language modeling instead of traditional left-to-right training?
A. It allows the model to capture more syntactic and semantic information from both directions of the sequence.
B. It reduces the computational cost of training by only processing part of the input sequence.
C. In this case, the model will be able to predict long-range dependencies across tokens.
D. It speeds up training by focusing on individual tokens instead of whole sentences.

Answer: A) It allows the model to capture more syntactic and semantic information from both directions of the sequence.
Explanation: Masked Language Modeling (MLM) allows the model to predict missing words using both the left and right context, giving it more complete information about the sequence than a traditional left-to-right model.

(18) Fine-tuning a pre-trained language model on a downstream task is a common approach. Which of the following is a key disadvantage of this fine-tuning method?
A. It requires a large labeled dataset for the specific task.
B. Fine-tuning often leads to catastrophic forgetting, where the model forgets its prior knowledge.
C. Fine-tuning is computationally inexpensive and can be done with small datasets.
D. Fine-tuning reduces the flexibility of the model to generalize to new tasks.

Answer: B) Fine-tuning often leads to catastrophic forgetting, where the model forgets its prior knowledge.
Explanation: Fine-tuning can lead to overfitting on the specific downstream task, and the model may forget the knowledge it learned during pretraining, especially if the fine-tuning data is small or not diverse enough.

(19) In fine-tuning a masked language model like BERT for a named entity recognition (NER) task, what is the primary reason why BERT's contextual embeddings are valuable for NER?
A. BERT provides a fixed representation for all tokens, which can be used across different tasks.
B. The pretraining task allows BERT to learn syntactic and semantic contexts, which is crucial for understanding entities in text.
C. Fine-tuning BERT on NER data significantly reduces the need for annotated data.
D. BERT’s encoder structure allows for easy extraction of specific entity types during fine-tuning.

Answer: B) The pretraining task allows BERT to learn syntactic and semantic context, which is crucial for understanding entities in text.
Explanation: BERT’s pretraining on a large corpus with a masked language modeling objective helps it learn rich, contextual embeddings that are particularly useful for downstream tasks like named entity recognition.

(20) Masked Language Models like BERT have the advantage of capturing both local and global contexts. How does this ability influence their performance in tasks such as question answering?

A. It allows the model to generate better answers by using information only from the immediate context of the question.
B. It helps the model use both the question and the surrounding context to understand and extract the relevant answer.
C. It allows the model to sort irrelevant sentences in the context of the question according to their importance.
D. It makes the model more efficient by focusing on local word interactions.

Answer: B) It helps the model use both the question and the surrounding context to understand and extract the relevant answer.
Explanation: The bidirectional context in models like BERT allows them to leverage both the question and surrounding context to better understand and answer questions, making them particularly effective for tasks like question answering.

(21) In the CKY algorithm, what is the primary reason why it can only be used with grammars in Chomsky Normal Form (CNF)?
A. CNF ensures that all productions have exactly two non-terminal symbols, making it easier to combine results in a dynamic programming table.
B. CNF guarantees that all rules produce terminal symbols, reducing ambiguity in parsing.
C. The CKY algorithm requires that all rules be deterministic, which CNF ensures.
D. CNF reduces the number of recursive calls needed for parsing.

Answer: A) CNF ensures that all productions have exactly two non-terminal symbols, making it easier to combine results in a dynamic programming table.
Explanation: CYK relies on breaking down the sentence into smaller parts, and CNF ensures that every rule has exactly two non-terminals or one terminal, simplifying the parsing process using dynamic programming.

(22) Constituency parsing aims to assign hierarchical structure to sentences. Which of the following is a primary limitation of context-free grammars (CFGs) for modeling natural language?
A. CFGs cannot represent dependencies between non-adjacent words.
B. CFGs require a large number of production rules to account for the variety of natural language structures.
C. CFGs are not efficient enough for parsing large corpora.
D. CFGs cannot generate sentences with multiple possible parses.

Answer: A) CFGs cannot represent dependencies between non-adjacent words.
Explanation: CFGs are limited in their ability to capture long-range dependencies between words, which are often crucial for natural language processing tasks.

Exercise 2: 6 points

Transformers are the backbone of modern natural language processing (NLP) models, powering applications like machine translation, text generation, and question answering. In this exercise, you will explore the multi-head attention mechanism in transformers and apply your knowledge to a masked language modeling task, analyzing various decoding strategies and their impact on predictions.

Part 1: Understanding Multi-Head Attention in Transformer Architecture

Assume a transformer model with the following specifications. Note: For simplicity, the batch dimension is not considered in this exercise. All shapes and calculations are based on sequence length and embedding dimensions only.

- Maximum Sequence Length: 20 tokens

- Embedding Dimension: 512
- Number of Attention Heads: 8

a. For the Full Multi-Head Attention Layer:

1. What are the shapes of the input and output matrices in the multi-head attention layer? Provide a clear explanation of how these shapes are derived.
2. What are the shapes of the Query (Q), Key (K), and Value (V) matrices after applying the linear transformations in the multi-head attention layer?
3. Write the theoretical equation for computing self-attention as a function of the Query (Q), Key (K), and Value (V) matrices. Explain each term in the equation.
4. Calculate the total number of trainable parameters in the multi-head attention layer. Provide a detailed explanation of your calculations, specifying the dimensions of each weight matrix.

b. For a Single Attention Head Within the Multi-Head Attention Layer:

5. After applying the linear transformations (projections), what are the dimensions of the Query (Q), Key (K), and Value (V) matrices for a single attention head? Explain your reasoning based on the embedding dimension and the number of attention heads.
6. For a single attention head, what is the shape of the attention matrix? Provide a detailed explanation of how this shape is determined during the computation of self-attention.
7. Derive the shape of the output matrix for a single attention head. Justify your answer using the self-attention computation and the model's specifications.
8. How can we obtain the final output shape from the outputs of individual attention heads in the multi-head attention layer? Provide the operations involved and the detailed shapes at each step.

Part 2: Masked Language Modeling

Given the sentence: "The dog [MASK] the ball" and the following probability distribution for the masked token (assume this distribution is the output of a SoftMax layer, representing the model's predicted probabilities for each token):

- catches: 0.45
- throws: 0.3
- chases: 0.1
- bites: 0.03
- eats: 0.02
- sleeps: 0.019
- hates: 0.015
- takes: 0.001

Questions

1. What are the tokens that will be retained when using greedy decoding? Justify your answer.
2. What are the tokens that will be retained when using top-k sampling with k=3? Justify your answer by explaining how top-k sampling works and why it retains the selected tokens.
3. What are the tokens that will be retained when using top-p (nucleus) sampling with p=0.92? Justify your answer by explaining the steps involved in top-p sampling and why it retains the selected tokens.
4. What are the tokens that will be retained when combining both top-p (p=0.92) and top-k (k=3) sampling? Justify your answer.
5. Recalculate the probabilities using temperature scaling with T=0.5 and explain its effect. The logits (pre-SoftMax values) for the tokens are as follows:

- catches:0.85
- throws:0.5
- chases:0.1
- bites: −0.3
- eats: −0.7
- sleeps: −0.8
- hates: −1.0
- takes: −2.0

Detailed solution:

Part 1: Understanding Multi-Head Attention in Transformer Architecture

Assume a transformer model with the following specifications:

- Maximum Sequence Length: 20 tokens
- Embedding Dimension: 512
- Number of Attention Heads: 8

a. For the Full Multi-Head Attention Layer

1. Shapes of Input and Output Matrices

Input Shape:

- Sequence Length $= 20$, Embedding Dimension $= 512$.
- Shape: 20×512.

Output Shape:

- The output shape matches the input: 20×512.

Explanation:

This is because the multi-head attention mechanism processes the input sequence and produces a new sequence of embeddings with the same dimensionality.

2. Shapes of Query (Q), Key (K), and Value (V) Matrices

- Each of $Q, K,$ and V matrices are obtained via linear transformation.
- Input Shape: 20×512.
- Output Shape of Q, K, V: 20×512.

3. Theoretical Equation for Self-Attention

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Explanation of Terms:

- Q: Query matrix (20×512).
- K: Key matrix (20×512).
- V: Value matrix (20×512).
- d_k: Dimensionality of each key vector. For a single head, $d_k = \frac{512}{8} = 64$.
- SoftMax: Ensures attention weights are normalized.

4. Total Trainable Parameters in Multi-Head Attention

- Each attention head has separate weight matrices for $Q, K,$ and V, and a linear transformation for output.
- Weight Matrices:
  - Q, K, V: Each 512×512.

- Output: 512×512.

- Parameters Calculation:

$$Q, K, V \quad : 3 \times (512 \times 512) = 786{,}432$$
$$\text{Output} \quad : 512 \times 512 = 262{,}144$$
$$\text{Total} \quad : 786{,}432 + 262{,}144 = 1{,}048{,}576.$$

b. For a Single Attention Head Within the Multi-Head Attention Layer

5. Dimensions of Q, K, and V Matrices

- Each attention head receives a fraction of the embedding dimension.
- Dimension per head: $\frac{512}{8} = 64$.
- Shape: 20×64.

6. Shape of the Attention Matrix

- Attention matrix is computed as QK^T.
- Shape: 20×20.
- Explanation: Each token attends to every other token (sequence length = 20).

7. Output Shape for a Single Attention Head

- Output of self-attention: 20×64.
- Explanation: Result of SoftMax$(QK^T)V$ retains sequence length and head-specific dimension.

8. Combining Outputs of Individual Heads

- Concatenate outputs from all heads: $20 \times (8 \times 64) = 20 \times 512$.
- After concatenation, the combined matrix of shape (20,512) is passed through a final learned linear transformation. This is done using a weight matrix Wo of shape (512,512), which maps the concatenated output back to the original embedding dimension.
- The resulting matrix after this transformation retains the shape: (20,512)

Part 2: Masked Language Modeling

Given Sentence: "The dog [MASK] the ball"
Probability Distribution:
catches: 0.45, throws: 0.3, chases: 0.1, bites: 0.03, eats: 0.02, sleeps: 0.019, hates: 0.015, takes: 0.001.

1. Greedy Decoding

The token with the highest probability is **"catches"** (0.45). Therefore, **"catches"** will be retained using greedy decoding.

2. Top-k Sampling (k=3)

- **Top-k sampling** involves selecting the top k tokens with the highest probabilities and then sampling from them. **k = 3** means we will retain the top 3 tokens with the highest probabilities.
- Top 3 tokens by probability: **catches, throws, chases**.

3. Top-p (Nucleus) Sampling (p=0.92)

- **Top-p (nucleus) sampling** involves selecting the smallest set of tokens whose cumulative probability is at least p. We include tokens from the highest probability down until their cumulative probability exceeds p.
- Cumulative probabilities:
  - catches: 0.45 (Cumulative: 0.45)

- throws: 0.3 (Cumulative: 0.45 + 0.3 = 0.75)
- chases: 0.1 (Cumulative: 0.75 + 0.1 = 0.85)
- bites: 0.03 (Cumulative: 0.85 + 0.03 = 0.88)
- eats: 0.02 (Cumulative: 0.88 + 0.02 = 0.90)
- sleeps: 0.019 (Cumulative: 0.90 + 0.019 = 0.919)
- hates: 0.015 (Cumulative: 0.919 + 0.015 = 0.934)
- takes: 0.001 (Cumulative: 0.934 + 0.001 = 0.935)

- Retained Tokens: **catches, throws, chases, bites, eats, sleeps**, **hates**

4. Combined Top-p (p=0.92) and Top-k (k=3)

- When combining **top-p sampling** with **top-k sampling**, both conditions need to be satisfied: Intersection of top-p and top-k results: **catches, throws, chases**.

5. Temperature Scaling (T=0.5)

- Formula: $p'_i = \frac{\exp(\log p_i/T)}{\sum_j \exp(\log\ p_j/T)}$.
- Adjusted Probabilities Calculation:

  Adjusted logits: Multiply each logit by $1/T = 2$.
  - catches: 1.7, throws: 1.0, chases: 0.2, bites: -0.6,
  - eats: -1.4, sleeps: -1.6, hates: -2.0, takes: -4.0.

  Adjusted probabilities
  - catches: **0.5181**
  - throws: **0.2573**
  - chases: **0.1156**
  - bites: **0.0519**
  - eats: **0.0233**
  - sleeps: **0.0191**
  - hates: **0.0128**
  - takes: **0.0017**

Explanation:

Lower values of T (e.g., T=0.5) sharpen the probability distribution. This results in:

- **Increased confidence** for the most likely tokens (e.g., **"catches"**), making them more likely to be selected.
- **Decreased probability** for less likely tokens (e.g., **"takes"**), making them much less likely to be selected.

This sharper distribution leads to a more deterministic outcome, where the most probable token is favoured.

Exercise 3: **4.5 points**

Model Overview:

In this exercise, we use a **Skip-Gram model with negative sampling** to predict context words for a given target word (center word). The architecture is designed as follows:

1. **Inputs**:
   - **Word Input**: A single integer (index) representing the target word (center word).
   - **Context Input**: A single integer (index) representing the context word.
2. **Embedding Layers**:
   - The model has two embedding layers:
     - **Word Embedding Layer**: This layer maps the target word (input) to a dense vector representation with dimensionality **Embedding dimension**.
     - **Context Embedding Layer**: Similarly, this layer maps the context word to a dense vector with the same **Embedding dimension**.

3. **Dense Layer**:
    - Both the **word embedding vector** and the **context embedding vector** are projected using dense layers to a smaller dimension of **Projected dimension**. This transformation reduces the dimensionality of both vectors before calculating the dot product.
4. **Dot Product**:
    - After the dense projections, the model computes the **dot product** between the two transformed embedding vectors. The dot product reflects how similar the target and context words are in the embedding space.
5. **Output**:
    - The dot product result is passed through a **sigmoid activation function**, which outputs a probability (between 0 and 1).

Given Parameters:

- **Window size**: 2.
- **Vocabulary size**: 5000.
- **Embedding dimension**: 128.
- **Projection dimension**: 10.

Questions:

1. List all the positive training instances that can be generated from the sentence "Karim was very smart" (don't use indices use actual words). **1 point**
2. List all the negative training instances that can be generated from the same sentence. **1 point**
3. How many weights are there in the Word Embedding and Context Embedding layers combined? **0.5 points**
4. How many total weights are there in the entire model? **1 point**
5. What is the appropriate loss function for the Skip-Gram model? Justify your answer. **1 point**

Solution:

Question 1:

The positive training pairs are generated by pairing each word with its context words within a distance of 2.
Replacing indices with words:
Positive pairs = [(Karim, was, 1), (Karim, very, 1), (was, Karim, 1), (was, very, 1), (was, smart, 1), (very, Karim, 1), (very, was, 1), (very, smart, 1), (smart, was, 1), (smart, very, 1)]

Question 2:

The negative training pairs. Replacing indices with words:
Negative pairs = [(Karim, Karim, 0), (smart, smart, 0), (was, was, 0), (Karim, smart, 0), (very, very, 0), (smart, Karim, 0)]

Question 3: How many weights are there in the Word Embedding and Context Embedding layers combined?

Step 1: Calculate the size of each embedding layer.

- Vocabulary size = 5000
- Embedding dimension = 128

Each embedding layer (Word Embedding and Context Embedding) has weights of size:
5000×128

Step 2: Total weights in both layers.

Since there are two embedding layers (Word and Context):
2× (5000×128) =1,280,000

Total weights: 1,280,000.

Question 4: How many total weights are there in the entire model?

Step 1: Weights in the embedding layers:

From Question 3, the Word Embedding and Context Embedding layers have 1,280,000 weights combined.

Step 2: Weights in the dense projection layers:

- Projection dimension = 10
- Embedding dimension = 128

Each dense layer projects from 128 to 10, so the weights in one dense layer:
128×10=1280
Additionally, there are biases in the dense layer (10 biases per layer):
For both Word and Context dense layers:
2× (1280+10) =2×1290=2580
Step 3: Total weights in the entire model:
1,280,000+2580=1,282,580
Total weights: 1,282,580.

Question 5: What is the appropriate loss function for the Skip-Gram model? Justify your answer.
Answer:
The appropriate loss function for the Skip-Gram model with negative sampling is the **binary cross-entropy loss**.
Justification:

- **Positive pairs:** For a target-context pair, the model should predict a high probability (close to 1) that the context word appears around the target word.
- **Negative pairs:** For a target-random word pair, the model should predict a low probability (close to 0).
- Binary cross-entropy is suitable because it measures the difference between predicted probabilities and actual labels (1 for positive pairs, 0 for negative pairs).

Exercise 4: 4 points

Consider the following Context-Free Grammar of a minute subset of French:

S -> NP VP | S PP
NP -> Det N | Det N PP | N
VP -> V NP | V NP PP | V PP
PP -> P NP
Det -> "le" | "la" | "un" | "une"
N -> "chat" | "chien" | "fille" | "garçon" | "maison" | "parc"
V -> "voit" | "aime" | "regarde"
P -> "dans" | "avec"

1. Convert the Grammar into Chomsky Normal Form (CNF) explaining the different steps and what they produce. Respect all the conventions. **2 points**

   Eliminate Null Productions:
   There are no null productions in this grammar.

   Convert Unit Productions:
   There is only one unit production; it is NP -> N and it will be replaced as follows:

   NP -> "chat",
   NP -> "chien", etc.

   Convert rules whose right-hand side consists of more than 2 Symbols:
   We need to introduce new non-terminal symbols for each of the following rules:

   NP -> Det N PP
   VP -> V NP PP

   The conversion of these rules produces:
   NP -> X_1 PP
   X_1 -> Det N
   VP -> X_2 PP
   X_2 -> V NP

The CNF version of the grammar is now:

S -> NP VP
S -> S PP
NP -> Det N
NP -> X1 PP
X1 -> Det N
VP -> V NP
VP -> V PP
VP -> X2 PP
X2 -> V NP
PP -> P NP

NP -> "chat" | "chien" | "fille" | "garçon" | "maison" | "parc"
N -> "chat" | "chien" | "fille" | "garçon" | "maison" | "parc"
V -> "voit" | "aime" | "regarde"
P -> "dans" | "avec"
Det -> "le" | "la" | "un" | "une"

2. In the below table:
   a. Give the complete result of the CKY analysis of the sentence `"le chat voit un chien dans le parc"`. Use the same conventions that we have seen in the lectures and, in case there are different parse trees for the whole sentence, use as many versions of S at cell [0,n] as needed (i.e. S_1, S_2, S_3, etc.). **1 point**
   b. Using different colours for each parse tree (NOT using any colour red, pink, or close to them), indicate the parse tree starting from the S symbol at position [0,8]. Do not colour the same arc with different colours. **0.5 points**

Le	**chat**	**voit**	**un**	**chien**	**dans**	**le**	**park**
Det	NP, X_1	-	-	S	-	-	S_1, S_2
	N, NP	-	-	-	-	-	S
		V	-	X_2, VP	-	-	VP, X_2
			Det	NP, X_1	-	-	NP
				N, NP	-	-	-
					P	-	PP
						Det	NP, X_1
							N, NP

3. Let *n* be the number of words in a sentence. What are the following complexities of the CKY algorithm, giving a brief explanation for each:
   a. Worst-case time complexity.
   The time complexity of the CKY algorithm is **$O(n^3)$**, where `n` is the number of words in the sentence.

To fill in the table any cell for a span of length *k*, we look at all possible ways of combining pairs of smaller spans (sub-spans) and there are $n - k + 1$ ways for this. Since there are n^2 cells, the worst-case time complexity of the CKY algorithm is **$O(n^3)$**.
0.25 point

b. Space complexity.
Since there are **n^2** cells in the table for a sentence of n words, even with a maximum of 2 pointers that get added to any produced non-terminal in the table cells, the space complexity of the CKY algorithm is **$O(n^2)$**.
However, for an ambiguous grammar, the space complexity can grow significantly. Each span in the CKY table can contain multiple non-terminal symbols, and the number of entries can grow exponentially with the number of derivations.
0.25 point